

SYSTEEMANALYSE

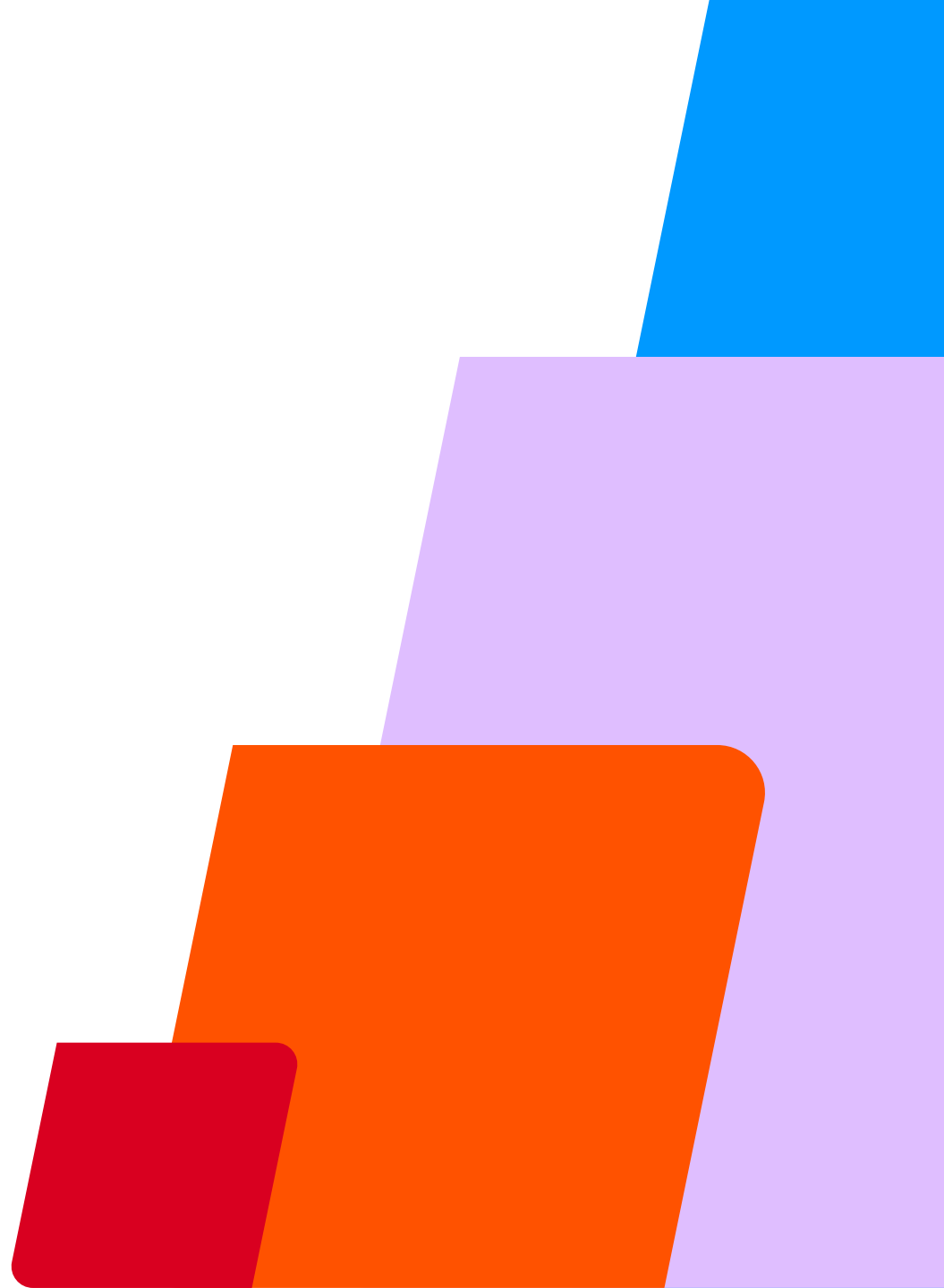
Belang en methodes van technische
systeemverkenning



LESDOEL

Je bent in staat om de opbouw van een systeem, en het proces door een bepaald systeem, in kaart te brengen en visualiseren, ook wanneer de documentatie van dit systeem beperkt is.

Je kunt ontwerpbeslissingen vastleggen in ADR's en het belang daarvan uitleggen.



AGENDA

- Wat is Software Architectuur?
- Belang & methodes architectuur analyse
- Visualisaties t.a.v. de opdracht
- **Architectural Decision Records**



SOFTWARE ARCHITECTUUR

WAT IS SOFTWARE ARCHITECTUUR?

ER IS NIET ÉÉN DEFINITIE...

ISO/IEC/IEEE 42010:2011

Formeel/structureel

"Fundamental concepts or properties of a system in its environment embodied in its elements, relationships, and in the principles of its design and evolution."

Ralph Johnson (GoF)

Filosofisch/sociaal

"Architecture is about the important stuff. Whatever that is."

Barry O'Reilly (Residues: Time, Uncertainty, and Change in Software Architecture.)

Praktisch/gedragsgericht

"Architecture is the structure of a software application that drives the behavior of a software application in its environment. That is, the way the system responds to scale, load, changes, integrations, and things we normally refer to as non-functional concerns."

WAT ZEKER IS....

Ieder software systeem heeft een architectuur

- met eigenschappen en gedrag
- met componenten en samenhang
- voortkomend uit beslissingen
- die soms onzichtbaar is

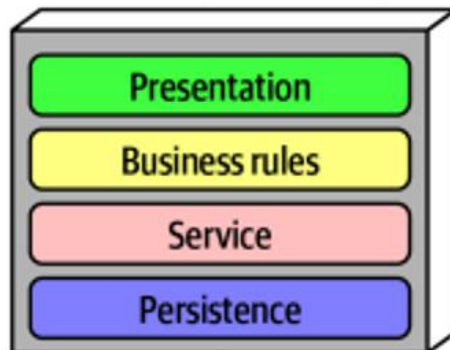
Ofwel....

*A level above data structure
and programming*

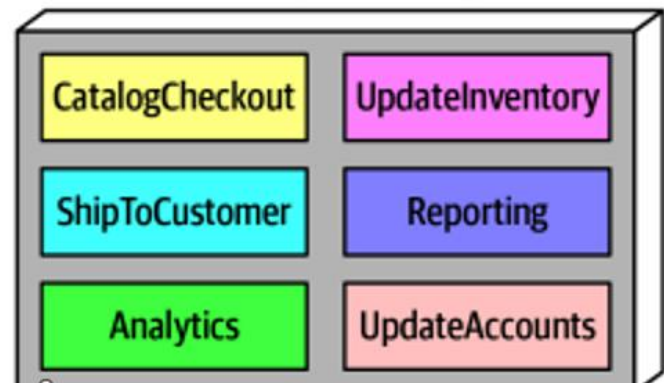
LATEN WE WEL ONDERSCHIED MAKEN TUSSEN...

Applicatie architectuur

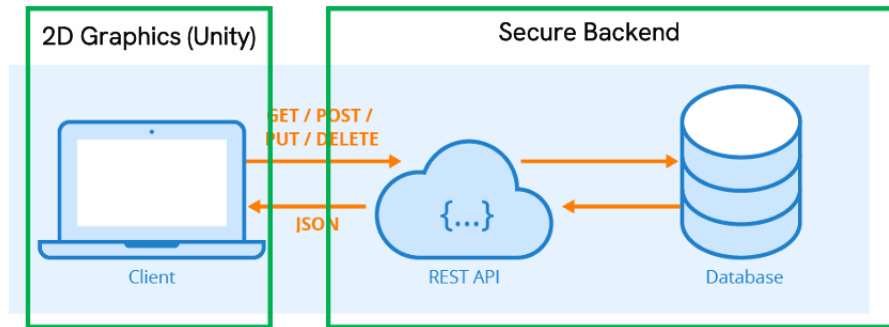
Technical partitioning



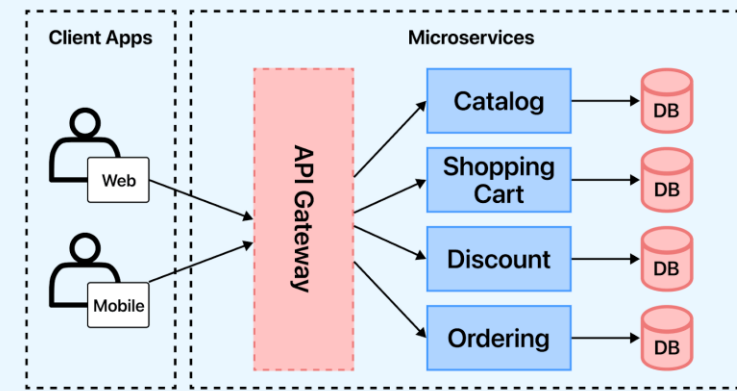
Domain partitioning



System architectuur (of system design)



API Gateway Microservice Design Pattern





ARCHITECTUURANALYSE

IN DE PRAKTIJK

"Ik wist niet dat die variabele ook ergens anders gebruikt werd." Je past een databaseveld aan omdat de naam niet logisch lijkt. Wat je niet wist: vijf andere modules lezen datzelfde veld. Na jouw wijziging crasht een scherm dat jij nooit hebt aangeraakt - en nooit hebt gezien, want je hebt het systeem niet verkend.

"Ik heb het gewoon opnieuw gebouwd, want ik begreep de bestaande code niet." Je krijgt een taak: voeg een notificatiefunctie toe. De codebase is groot en onbekend. Je schrijft een nette klasse - totdat je begeleider aangeeft dat er al een `service` bestaat, drie mappen dieper. Jouw versie conflicteert ermee. Twee uur debuggen later begrijp je waarom het systeem zich raar gedroeg

WAAROM ARCHITECTUURANALYSE?

Een **Software Engineer** die een bestaand systeem niet analyseert voordat hij begint, ontwerpt in een vacuüm. Het gevolg is:

- keuzes die eerder gemaakte lessen schenden
- toename *technical debt*

Je enthousiasme om te gaan *bouwen*, kan resulteren in ondoordachte en inefficiënte oplossingen. Begrijp het systeem (en het bijbehorende ontwikkelproces) voor je start.

Op je opleiding bouw je een applicatie op vanaf nul.
Maar in een bedrijf stap je bijna altijd in een rijdende trein.

HOE ANALYSEER JE EEN SOFTWARE ARCHITECTUUR?

METHODES ARCHITECTUURANALYSE

Praktische verkenning

- **Vragen stellen** aan een *software engineer* of *architect*
- **Documentatie** lezen (wiki's, readme, changelogs)
- **CI/CD pipelines** bekijken
- **Code** analyseren (mappen, packages, folders)

AI-ondersteuning

Voor het analyseren van een bestaande codebase kan gebruik worden gemaakt van **DeepWiki** (deepwiki.org). DeepWiki genereert van elke publieke repository een documentatiebestand in wiki-vorm, waarmee snel inzicht verkregen kan worden in de opbouw en lagen van een softwaresysteem.

OPDRACHT

Praktische verkenning OpenMRS

+/- 10 minuten

Voer met je groep een praktische verkenning van OpenMRS uit.

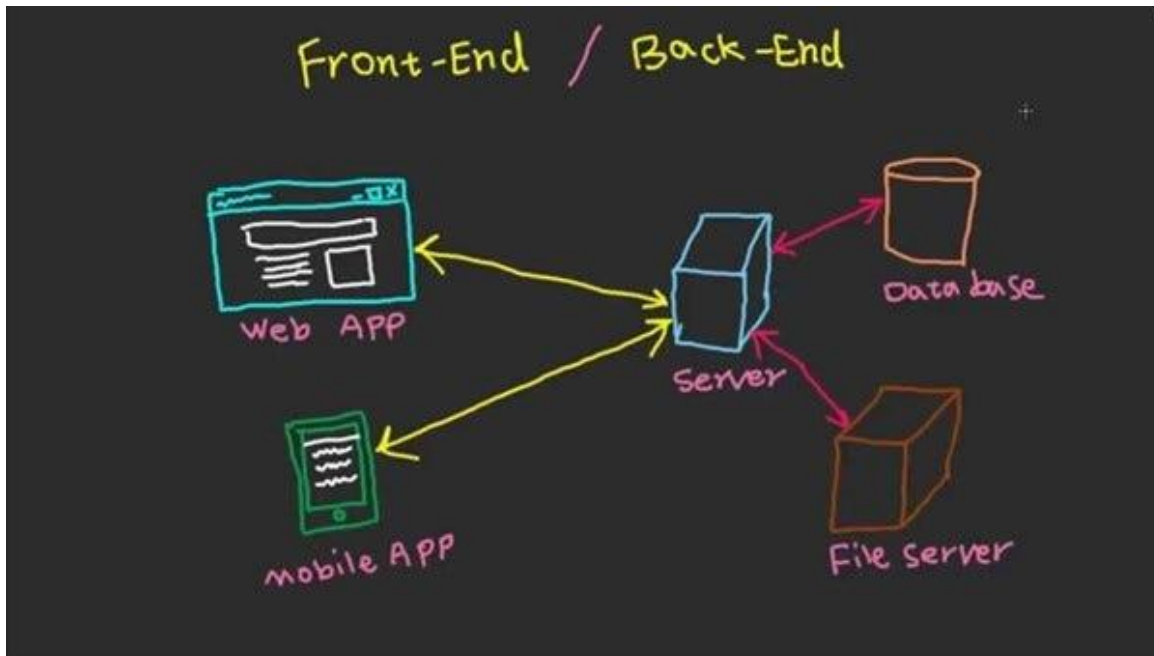
Denk aan

- Welke technologieën worden gebruikt?
- Kun je iets zeggen over de applicatie architectuur?
- Kun je iets zeggen over de systeem architectuur?
- Kun je informatie vinden over de ontwerpbeslissingen?

Gebruik dit als input voor je architectuurvisualisatie later

BEKIJK....

Een centrale server vormt het hart van het systeem en fungeert als enig aanspreekpunt voor alle clients — een web-app en een mobiele app sturen requests naar de server en ontvangen daarvan de responses terug. De server zelf communiceert aan de achterkant met twee opslagcomponenten: een database voor gestructureerde data en een files server voor bestandsopslag. Alle logica en toegangscontrole loopt via de server als centrale tussenpersoon.



WAAROM VISUALISEREN?

Een visualisatie helpt...

- ... bij het **inzichtelijk** maken wat de grenzen zijn van het systeem
- ... bij het bieden van (essentiële) documentatie van een systeem- of applicatiearchitectuur voor andere software engineers
- ... als tekenbord waar toekomstige wijzigingen op kunnen worden getoetst

Teken het voor je begint te realiseren!

WAARAAN VOLDOET EEN GOEDE VISUALISATIE?

Het doel van een visualisatie is om inzicht te geven.

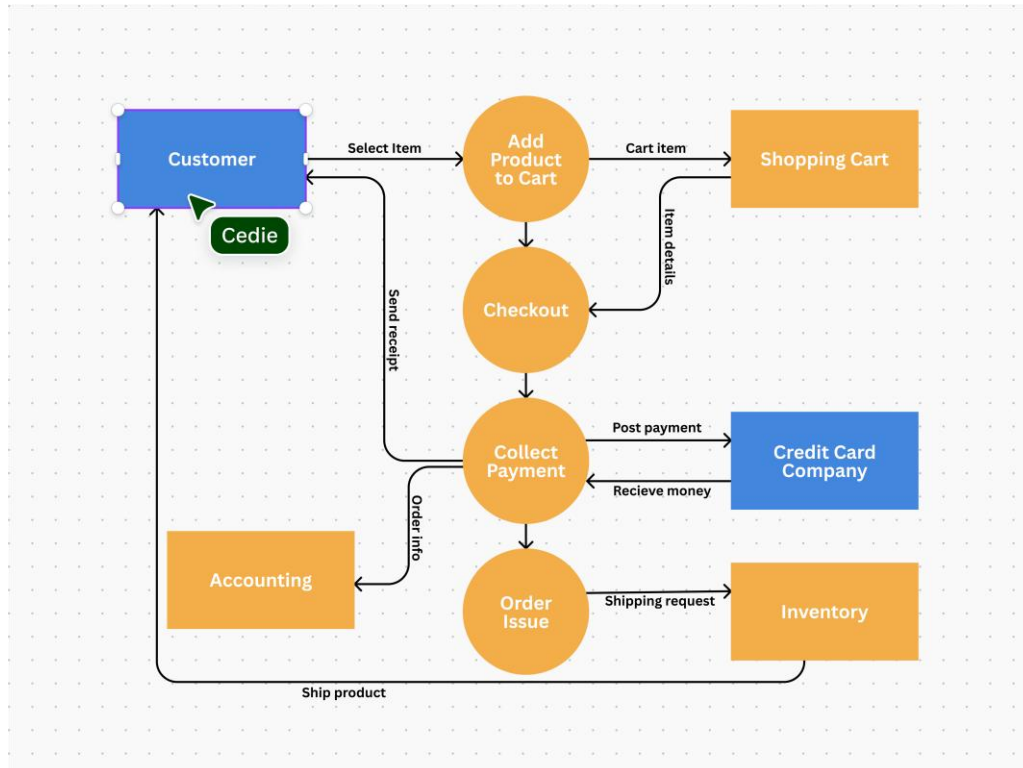
Het is **precies genoeg** - niet elke klasse, maar de componenten die er architecturaal toe doen.

Het is **geadresseerd aan iemand** - een diagram voor een andere software engineer ziet er anders uit dan een diagram voor een opdrachtgever.

Tip: teken niet alleen lijnen, maar benoem ook wat de lijnen doen of betekenen voor de geadresseerde (welke actie zit eronder? Welk protocol gebruikt de verbinding?)

SOORTEN VISUALISATIES

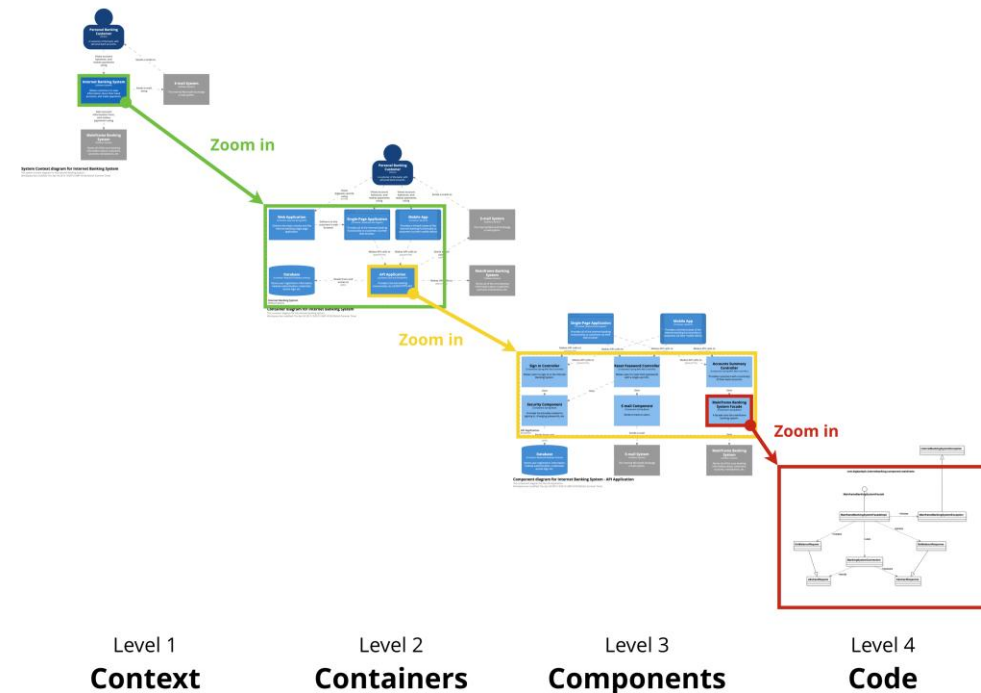
Wat gebeurt er als X plaatsvindt?



Voorbeeld: Data Flow Diagram (DFD)

Alternatief: Sequentiediagram

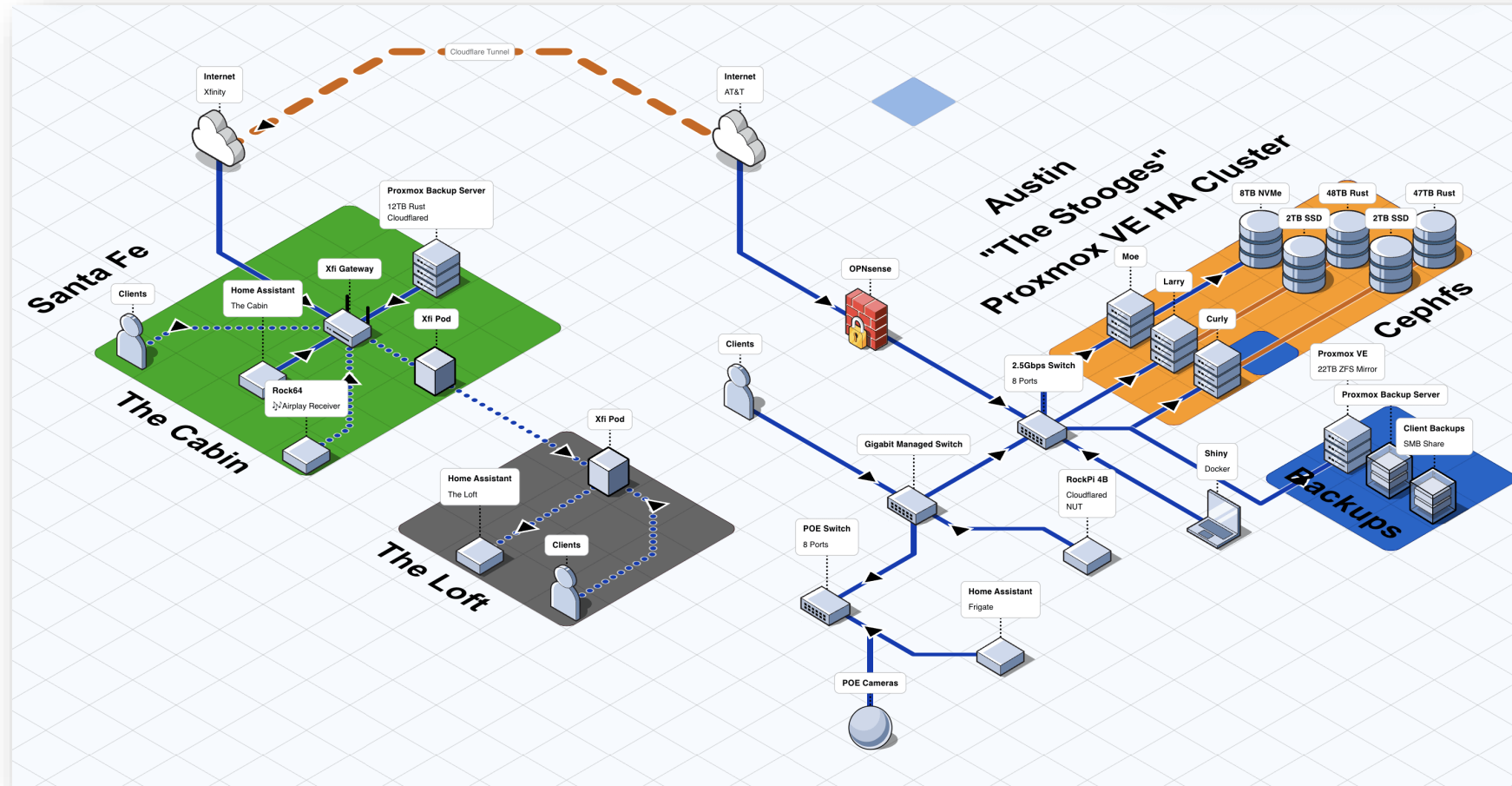
Hoe is het systeem opgebouwd?



Voorbeeld: C4 model

Alternatief: 4 + 1 model, UML component diagram

SOORTEN VISUALISATIES



Voorbeeld: Applicatielandschap, systeemlandschap

Tool: FossFLOW

APPLICATIE INTEGRATIE



WAAROM APPLICATIE- INTEGRATIE?

Organisaties gebruiken meerdere systemen (ERP, CRM, webapps)

Bedrijfsprocessen lopen over systemen heen

Data zit verspreid

Systemen “praten” niet goed met elkaar

Hoe laten we systemen effectief samenwerken?

PROBLEMEN EN BELEMMERINGEN

Typische problemen

- Data-silo's: data die geïsoleerd is en lastig bereikbaar of deelbaar
- Dubbele data
- Handmatige koppelingen, bv via imports van Excelbestanden

Gevolg

- Fouten
- Inefficiëntie
- Lastig in onderhoud

POINT TO POINT KOPPELING

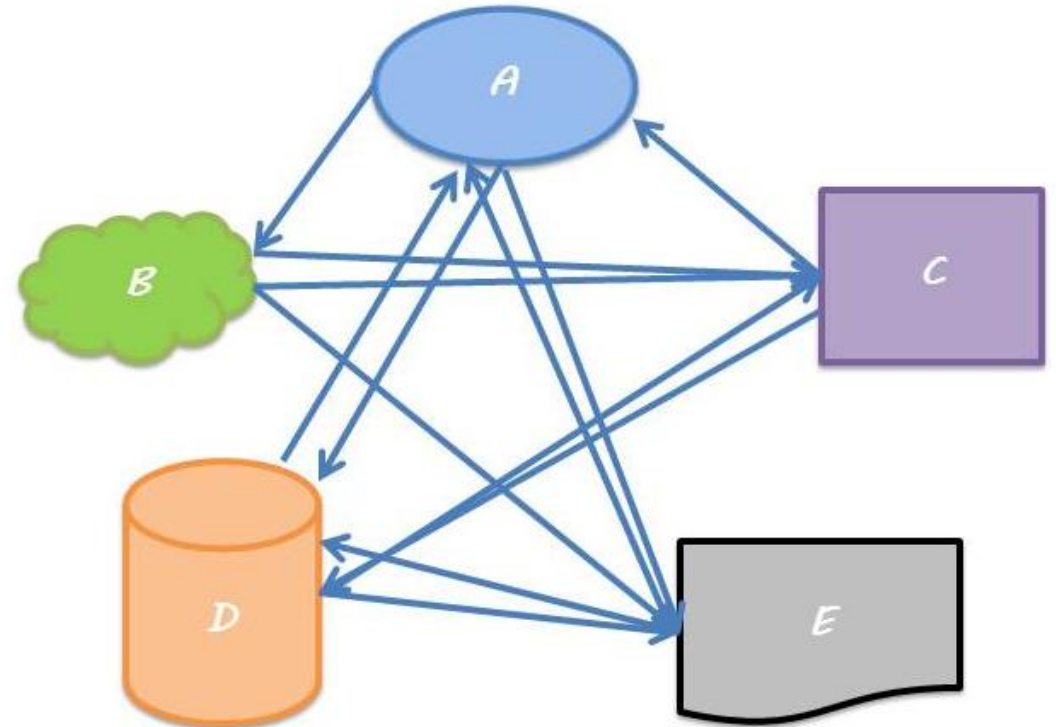
Koppelt systemen aan elkaar met directe verbindingen

Voordelen

- Snel te realiseren
- Simpel (op het eerste gezicht)

Nadelen

- Explosie van koppelingen
- Sterke afhankelijkheden = tight coupling
- Niet schaalbaar



Kunnen we functionaliteit loskoppelen van systemen?

→ **Denk in services in plaats van systemen**

SERVICE ORIENTED ARCHITECTURE

Service Oriented Architecture

Architectuurstijl waarbij functionaliteit wordt aangeboden via *herbruikbare services*.

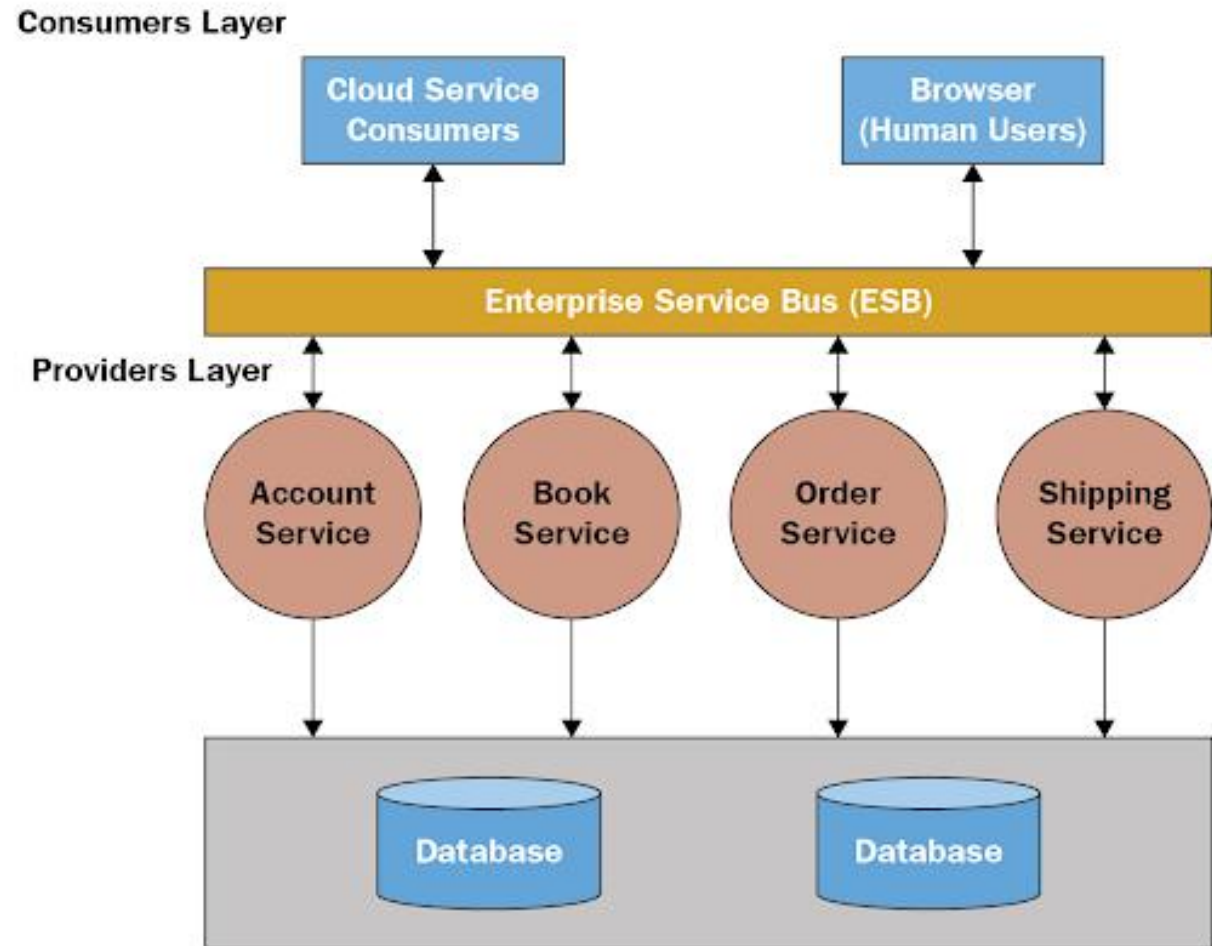
- Loose coupling
- Services en protocollen met duidelijke interfaces
- Herbruikbaarheid

Voorbeelden?

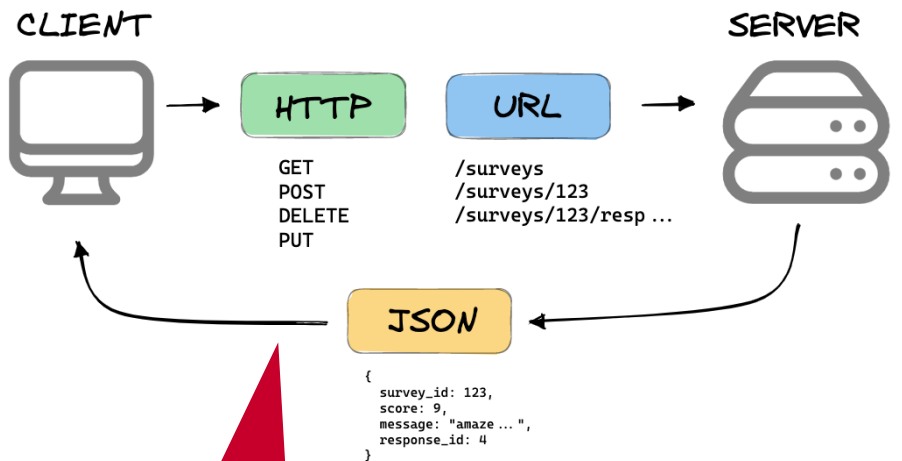
SERVICE ORIENTED ARCHITECTURE

Services met gescheiden, *dedicated* deel van functionaliteit van het totale systeem.

ESB maakt integratie toegankelijker maar ook veel complexer

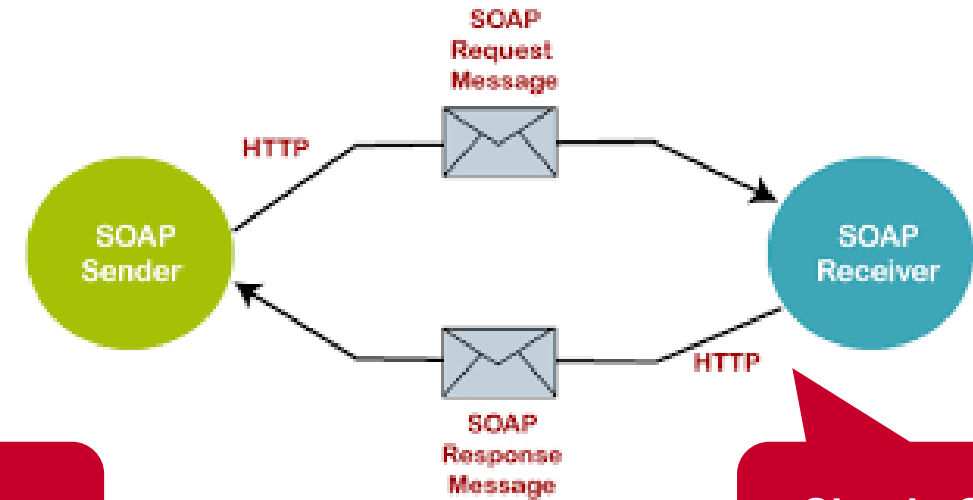


COMMUNICATIEMETHODEN



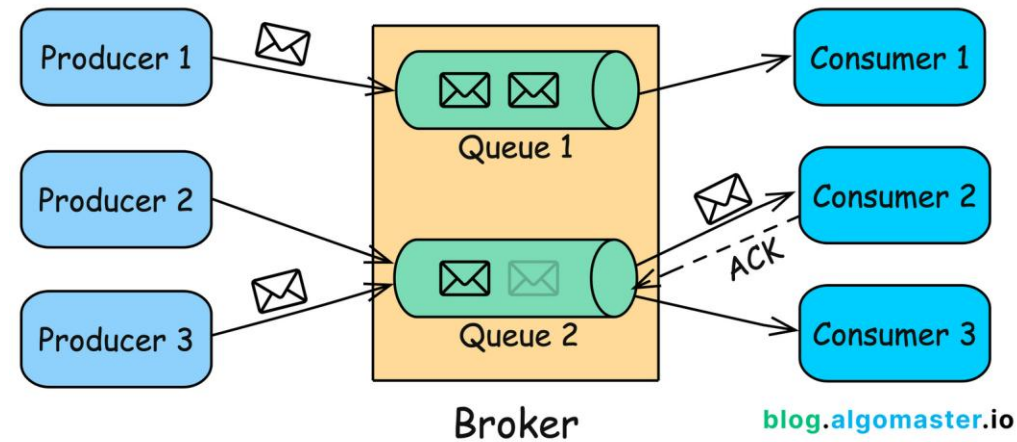
RESTful
service
interfaces

mannhowie.com



Messaging
via queues

Simple Object
Access Protocol





DE OPDRACHT

OPDRACHT

Bekijk de opdracht op Brightspace

+/- 10 minuten

Bekijk met je projectgroep de opdracht op Brightspace

Noteer

- Welke visualisaties horen hierbij?
- Staan er termen in die onduidelijk zijn?
- Welke non-functional requirements gaan leiden tot een architectuur ontwerpbeslissing



HISTORY

ARCHITECTURAL DECISION RECORDS

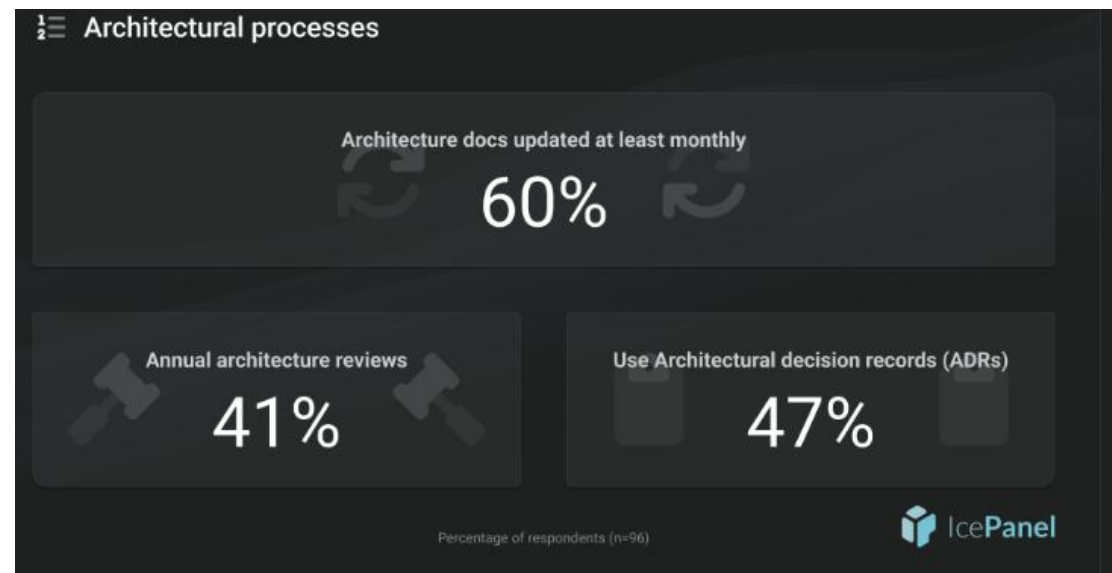
ONTWERPBESLISSINGEN HERLEIDEN

Iedere beschrijving (of visualisatie) is vastgelegd op één moment in de tijd. De architect die drie jaar geleden de frameworkkeuze maakte, werkt er niet meer. Wat nu?"

- User stories?
- Epics?
- Iemands geheugen?
- `git log --follow -p - readme.md`

ARCHITECTURAL DECISION RECORDS

- Beschreven door Michael Nygard in 2011 als praktische manier om architectuur ontwerpbeslissingen vast te leggen
- Verschillende templates beschikbaar



ADR: NYGARD ADR



Decision record template by Michael Nygard

This is the template in [Documenting architecture decisions - Michael Nygard](#). You can use [adr-tools](#) for managing the ADR files.

In each ADR file, write these sections:

Title

Status

What is the status, such as proposed, accepted, rejected, deprecated, superseded, etc.?

Context

What is the issue that we're seeing that is motivating this decision or change?

Decision

What is the change that we're proposing and/or doing?

Consequences

What becomes easier or more difficult to do because of this change?

Status is een onderschat veld: "Proposed", "Accepted", "Deprecated", "Superseded". Een ADR mag verouderd raken

ADR: MARDKOWN ADR (MADR)



Title

Context and Problem Statement

Decision Drivers

Considered Options

Decision Outcome

Consequences

Option 1

Good because {argument}

Neutral, because {argument}

Bad, because {argument}

More information

ADR: MARKDOWN ADR (MADR)

status: Accepted

date: 2022-11-22

deciders: ZIO

AD: System Decomposition into Logical Layers

Context and Problem Statement

Which concept is used to decompose the system under construction into logical building blocks?

Decision Drivers

- * Desire to divide the overall system into manageable parts to reduce complexity

- * Ability to exchange system parts without affecting others

Considered Options

1. Layers pattern
2. Pipes-and-filters
3. Workflow

Decision Outcome

We decided to apply the Layers pattern and neglected other decomposition pattern such as pipes-and-filters or workflow because the system under construction and its capabilities do not suggest an organization by data flow or control flow. Technology is expected to be primary driver of change during system evolution.

Consequences

- * Good, because the Layers pattern provides high flexibility regarding technology selections within the layers (changeability) and enables teams to work on system parts in parallel.
- * Bad, because there might be a performance penalty for each level of indirection and some undesired replication of implementation artifacts.

More Information

- * The three decomposition options come from the Cloud Computing Pattern [Distributed Application] (https://www.cloudcomputingpatterns.org/distributed_application/).

- * The Layers pattern is featured in POSA Volume 1, see [<http://www.dre.vanderbilt.edu/~schmidt/POSA-tutorial.pdf>](http://www.dre.vanderbilt.edu/~schmidt/POSA-tutorial.pdf)

A follow-on decision will be required to assign logical layers to physical tiers.

OPDRACHT

Schrijf een eerste ADR

+/- 15 minuten

De opdracht vraagt je om een aparte service te ontwikkelen. Een losstaand fysiek component.

Stel een eerste ADR op die deze keuze voor jullie verantwoord en neem de ADR later op in je code-base.

SPRINT DOEL

Zie Brightspace